WHITEPAPER / PROPOSED SPECIFICATION

Agentic Architecture Blueprint (AAB) 0.1

Architecture Contracts, Authorized Baselines, and a Reference Validator

Human-readable architecture. Authenticated approval. Independent verification.

Technical Whitepaper — Revised Discussion Draft

12 October 2026 • Whitepaper version 2 (corrected edition)
Proposed specification version 0.1

Abstract

A vendor-neutral proposal for describing architectural intent, protecting approved design decisions, and independently verifying explicitly testable architectural constraints.

Contents

Status. Whitepaper v2; the proposed specification version remains 0.1. This is a discussion draft, not a published standard or a finished product. Examples, commands, schemas, and evaluation thresholds describe a proposed design. Compatibility with ISO/IEC/IEEE 42010 is an objective, not a claim of certification.

Executive summary

AI coding agents make software faster to produce, but they can also make consequential design decisions while implementing apparently small requests. An agent may bypass a component boundary, introduce an undocumented integration, add a directory that existing architecture checks do not inspect, or quietly change the document intended to constrain its work. The new code can pass functional tests while the long-term system architecture becomes less coherent. The issue is not unique to AI, but agentic speed, autonomy, and repeated context resets can amplify it [14].

The **Agentic Architecture Blueprint (AAB)** is a proposed way to make approved architecture a first-class engineering contract. It retains understandable architectural views, provides a small machine-readable vocabulary for important rules, and requires independent evidence of what was actually checked. AAB builds on the 4+1 view model, ISO/IEC/IEEE 42010, architecture decision records, spec-driven workflows, and existing architecture-testing tools rather than replacing them [1, 2, 5-11].

AAB 0.1 has four central mechanisms:

1. **A canonical blueprint:** identified components, relations, architectural decisions, scenarios, and typed rules are linked to human-readable explanations.
2. **Authorized baselines:** approval is an independently verifiable signed attestation that binds a specific blueprint digest to an authorized decision. A mutable approval field or digest inside the blueprint is insufficient.
3. **Complete declared coverage:** eligible source and configuration artifacts must be classified as governed, explicitly excluded under an approved policy, or reported as unmapped. At the governed conformance level, unmapped artifacts block automatic acceptance.
4. **A reference validator with honest evidence:** supported checks report `conformant`, `violated`, `unverified`, or `not_applicable`; execution errors and coverage failures are distinguished from genuine passes. An L2 gate fails closed on unresolved blocking obligations.

Version 2 deliberately narrows the initial core to **three rule types** with bounded checking semantics: `forbidden_dependency`, `required_dependency`, and `configuration_policy`. Runtime network access and scenario-evidence rules remain important, but should be standardized only after their observation and coverage models can be specified with comparable precision.

AAB does not guarantee that a software system is architecturally correct. Its more defensible promise is that decisions become traceable, material changes require authorized approval, important properties can be checked reproducibly within a declared scope, and gaps in assurance remain visible. The first implementation should be evaluated against conventional prompting and existing spec-driven workflows using seeded violations, independent raters, and measured governance cost.

1. Introduction: software blueprints in the age of coding agents

A building design is more than an attractive drawing. It incorporates dimensions, materials, responsibilities, sign-offs, and inspections. The drawing communicates intent; engineering controls establish which departures are permitted and whether the resulting structure meets selected requirements.

Software architecture has long served a similar communication function. An architect might draw three

boxes - an API, a domain service, and a persistence adapter - and specify that incoming requests must pass through the domain layer before reaching storage. A human developer can understand the rationale. An AI coding agent can also understand it. Yet when asked to deliver an urgent feature, the agent may import the persistence adapter directly from the API. The feature works. The test suite passes. The design boundary has been weakened.

Now imagine several agents making similar local optimizations across weeks of development, each working with different context and different task instructions. A seemingly well-specified project can accumulate architectural drift without an obvious single failure. Research on AI coding agents has described the role of implicit architectural choices and prompt-dependent design outcomes; that work remains an emerging research area rather than proof that one remedy is best [14].

The central question is therefore **not** whether AI can follow architectural prose. It is whether the organization can preserve and verify significant design decisions **independently of the agent that writes the code**.

AAB proposes a practical answer: people and AI reviewers refine an architecture; authorized humans approve a fixed baseline; coding agents implement against it; a separately trusted validator checks the supported rules; and proposed architectural departures return to review. In short, **architecture becomes a governed interface between human intent and autonomous implementation**.

2. The problem: why existing practices do not fully close the loop

Four recurring problems motivate the proposal.

Ambiguous constraints. Statements such as “keep the domain isolated” or “never bypass security” express important intentions but are not sufficiently precise for independent checking. Different agents may interpret them differently.

Fragmented authority. Diagrams, architecture decision records (ADRs), task plans, agent instruction files, and CI configuration often contain overlapping rules. When they disagree, the binding version is unclear. Updating an instruction file does not necessarily update the architecture that reviewers approved.

Unreviewed change and self-governance. Agents can modify both code and repository guidance. A file containing an “approved” field or content digest is not protected merely because its contents are readable by CI. An agent able to edit both the rule and its approval metadata may effectively approve itself.

Incomplete verification. Checks commonly operate on only known packages or explicit path globs. A new directory can fall outside all of them. Moreover, finding no forbidden import in a static dependency graph says nothing conclusive about runtime reflection, dynamic loading, network calls, or paths the checker never examined.

There are excellent partial solutions. The 4+1 model and ISO 42010 structure architectural descriptions [1, 2]. ADRs preserve reasoning [5]. The C4 model and Structurizr provide model-based visualizations [6]. Spec Kit and OpenSpec structure changes and implementation work [7, 8]. ArchUnit and Conftest enforce defined properties [10, 11]. However, these do not automatically form a **shared, independently authorized architecture-conformance contract** across tools. AAB is an attempt to standardize that missing connection rather than claim the individual ingredients are new.

3. Design principles and terminology

The proposal is guided by five principles: **human authority over architecture**, **a canonical structured model**, **limited but precise machine-checkable obligations**, **independent enforcement**, and **portable evidence with explicit limits**. The standard must be vendor-neutral and implementable without a particular coding agent, language, or cloud service.

Terminology is deliberately controlled:

Term	Meaning in this proposal
Blueprint	Versioned package identifying architectural elements, views, decisions, and contracts.
Element	Stable-ID model entity, including a software component. `components` is the concrete element collection in the 0.1 example.
Contract	A named grouping of one or more binding architectural **rules**, with rationale and scope, declared under `contracts` in `blueprint.yaml` in AAB 0.1.
Rule	A single typed assertion with stable ID, enforcement level, and declared checking requirements. In the schema, rules are stored under `rules`.
Validator	The independently executed AAB CLI/library responsible for interpreting the package, orchestrating checks, and reporting outcomes.
Adapter (or **checker**)	A versioned implementation of a particular check (for example, an import-graph analyzer or structured-config evaluator). “Checker” describes its function; “adapter” describes its plugin interface.
Baseline	A content-addressed blueprint version bound to an authorization decision.
Approval attestation	Signed statement *outside the blueprint package* that names its digest and records a decision under a trusted authority policy.
Evidence	Structured observations about rules, coverage, execution, versions, and source revisions; not blanket proof of correctness.

Where conformance language is needed, the proposal uses **MUST**, **SHOULD**, and **MAY** in the sense of RFC 2119 and RFC 8174 [12]. Requirements are *proposed normative rules for AAB 0.1*, not requirements of an established international standard.

4. AAB architecture: 4+1 plus Contract and Evidence

AAB uses the **logical**, **process**, **development**, and **physical** views, connected through **scenarios** - Kruchten’s original 4+1 framing [1]. The logical view describes responsibilities; the process view covers concurrent and runtime behavior; the development view covers modules and dependency directions; the physical view covers deployment and trust boundaries; scenarios connect those descriptions to intended behavior. ISO/IEC/IEEE 42010 provides terminology for stakeholders, concerns, viewpoints, and correspondences between descriptions [2].

AAB adds two **cross-cutting layers**, not two competing views:

- **Contract:** the subset of architectural decisions expressed as typed, scoped, binding assertions and authorized exceptions.
- **Evidence:** recorded observations about which assertions were checked, over which artifacts, by which checker, under which baseline, and with which limitations.

Not every important architectural property belongs in the machine-checkable subset. A design rationale, trade-off, or judgment can remain prose if it cannot be tested honestly. Such statements still guide human review, but they must not be mislabeled as mechanically verified.

4.1 Canonical package and trust separation

A suggested logical representation is:

```
architecture/
  blueprint.yaml          # Core model; elements, rules, references
  views/                  # Human-readable 4+1 descriptions
  decisions/              # ADRs with stable IDs
  amendments/             # Proposed changes (not approvals)
AGENTS.md                 # Generated, not authoritative

# Stored outside agent-editable blueprint package:
trusted-approval-store/     # Signed baseline attestations
trusted-policy-store/       # Signer policy, CI gate configuration
ci-artifacts/               # Produced validation evidence
```

The **core model is the source of truth for element IDs, mappings, contract groupings, and rule fields**. In AAB 0.1, contracts are declared under `contracts` in `blueprint.yaml`; each rule refers to its contract by stable ID. View documents explain and reference that model; checker declarations are **inline in each rule**. There is no second authoritative `contracts/` or `verification/` directory in this core profile. A tool may generate adapter configuration in a build directory, but that generated configuration must identify the input blueprint digest.

The blueprint's canonical content digest should include the model and explicitly enumerated referenced files. The approval attestation and generated evidence must be external, avoiding circular hashes and preventing ordinary source changes from fabricating a new approval. A signed approval must be checked against a **trusted signer/identity allowlist and authorization policy** that the implementation agent cannot change.

4.2 Canonicalization and identities

AAB 0.1 should define a `manifest` consisting of canonical, relative paths and content hashes. A defensible implementation could first serialize a manifest as canonical JSON according to RFC 8785, then compute its SHA-256 digest [18]. The manifest would enumerate exactly which bytes are covered; JSON canonicalization alone does **not** canonicalize arbitrary YAML or Markdown file contents.

Conformance fixtures must define path normalization (for example, forward slashes and Unicode normalization), ordering, duplicate-path behavior, symlink policy, path traversal rejection, and how line-ending changes affect hashes. The initial reference validator SHOULD reject symlinks and paths outside the package unless the specification explicitly supports them. Rules and elements need stable IDs independent of filenames. CI must verify that a manifest cannot quietly omit a referenced normative artifact.

5. The AAB 0.1 core contract

5.1 Exactly three initial rule types

Rule type	Proposed semantics	Minimum required check
`forbidden_dependency`	In a resolved direct dependency graph, no edge may run from a mapped source element to a mapped target element.	The adapter must analyze all eligible source artifacts for both elements and report graph-construction limits.
`required_dependency`	At least one declared direct edge must exist from an applicable, nonempty source set to the target under the selected graph relation.	Missing/unresolvable sources or targets cannot produce a pass; the rule must not pass vacuously.

Rule type	Proposed semantics	Minimum required check
`configuration_policy`	A specified property of explicitly selected, parsed configuration resources must satisfy a typed predicate.	Each selected resource and property must resolve under a declared parser, selector, and evaluation policy.

For dependency rules, the first standardized relation is **direct static `imports`**, not "all runtime interactions." The adapter must specify how it treats aliases, re-exports, type-only imports, generated code, workspaces, and unresolved external modules. For configuration policies, 0.1 SHOULD start with a small predicate set such as `equals`, `not_equals`, `present`, or `absent`. Selectors that match no resources require an explicit rule-specific interpretation and must not silently count as success.

Two rule families are **deferred**:

- `restricted_runtime_access` needs an explicit observation domain (for example, parsed Kubernetes network policies or runtime egress telemetry). No generic proof of restricted runtime access is claimed.
- `scenario_evidence` requires more than discovering a tagged test. A future version must specify test identity, executable outcome, environment, coverage claim, and credible binding to a scenario.

Implementers may experiment with these as vendor extensions, but must not present extension results as AAB 0.1 core conformance.

5.2 Worked contract example

The following illustrates a proposed **single-file AAB core format**. It is not a claim that a published validator currently accepts it.

```yaml
schema_version: "0.1-draft"
blueprint_id: order-service
profile: AAB-L2

coverage:
  eligible_roots: ["src/**"]
  explicitly_excluded: []
  unmapped_policy: block

components:
  - id: api
    paths: ["src/api/**"]
  - id: domain
    paths: ["src/domain/**"]
  - id: persistence_adapter
    paths: ["src/adapters/persistence/**"]

contracts:
  - id: LAYER-001
    title: Layer isolation
    scope: [api, domain, persistence_adapter]
    rationale: Preserve separation of responsibilities.

rules:
  - id: ARCH-001
    contract: LAYER-001
    statement: >
      API MUST NOT directly import persistence adapters.
    type: forbidden_dependency
```

```
    source: api
    target: persistence_adapter
    relation: imports
    enforcement: blocking
    rationale: Preserve domain boundary separation.
    decision: ADR-0001
    verification:
      adapter: static-import-graph
      adapter_version: "1.x-pinned-by-lock"
      resolution_policy: strict

approval_policy_ref: arch-governance-policy-v1
```

`LAYER-001` is the declared contract grouping for `ARCH-001`; the validator MUST reject rule references to undefined contracts. `approval_policy_ref` is an **identifier of an externally trusted policy**. It is not a locally editable grant of approval, and the string alone does not authorize anyone. The adapter version string above indicates an intended pinned release; the actual lock record must resolve it to an exact version and digest before an L2 check is valid.

5.3 Close the unmapped-artifact loophole

The validator must account for the complete **declared eligible artifact universe**. An L2 check MUST enumerate tracked eligible source files in the candidate revision and additional generated/build outputs required by the applicable adapter policy. Each artifact must have exactly one primary component assignment, or an explicit, approved exclusion. Overlaps, duplicate assignments, unresolved glob patterns, excluded artifacts without approval, and eligible unmatched files require distinct diagnostics.

For example, `src/shortcuts/direct_db.py` matches the `eligible_roots` selection in the example, but matches none of the three declared components. The validator must return **`unmapped_artifact`** and prevent automatic L2 acceptance, even if `ARCH-001` finds no forbidden import among its mapped files. This closes an important bypass of path-based checks.

Files outside `eligible_roots`, ignored build products, generated code, vendored dependencies, symlinks, and workspace boundaries require explicit scoping rules. The claim is **complete accounting within a declared universe**, not automatic discovery of all code that might execute anywhere. A newly introduced eligible directory must never disappear from the conformance result merely because the model has not yet been updated.

6. Approval is an authenticated decision, not a hash

A digest identifies content; it does not confer permission. **Approval is a separate act by a designated authority.** The baseline must be bound to a verifiable statement signed under credentials whose trust anchors and authorization rules are controlled outside the agent-editable worktree.

AAB should support an in-toto Statement v1 shape, where the `subject` names the canonical blueprint-manifest digest and a versioned AAB predicate records the approval decision and policy context [19]. Sigstore/Cosign is one practical signing and verification mechanism; protected signed Git tags or an organizational signing service may also be acceptable implementations if the trust and authorization properties are equivalent [20].

A minimum approval predicate would include: `blueprint_id`, `manifest_digest`, `decision` (`approve` or `revoke`), `authority_policy_id`, `review_request_id`, `repository_scope`, `approved_at`, and where applicable `supersedes` or `expires_at`. **Signing alone is insufficient:** the validator must authenticate the signature, verify the permitted signer identity or certificate issuer against protected policy, check that the attestation subject matches the exact digest, and apply replay/revocation or supersession rules.

An agent may draft a new baseline and propose an amendment. It may **not** create a valid authority decision simply by modifying `blueprint.yaml`, editing a purported approval file, or recomputing a checksum. A trusted CI gate should obtain authorizations from an out-of-band or protected channel. Untrusted

pull-request code must not be allowed to replace the approval validator or the trust roots it uses.

6.1 The two-cycle workflow

Design cycle: stakeholders identify important concerns and scenarios; architecture and constraints are drafted; humans and AI reviewers challenge inconsistencies and omissions; authorized reviewers accept, revise, defer, or reject findings. A successful decision produces an externally attestable baseline.

Implementation cycle: agents receive the signed baseline and applicable instructions, plan code changes, and run preliminary checks. Trusted CI independently verifies the approved rules against the candidate revision. A violation is repaired or becomes a proposed architecture amendment. The agent never turns its own amendment into approval.

Exceptions should also be controlled: each needs a stable ID, a rule and scope reference, a named owner, reason, expiry or review date, and an independently authorized approval record. Unrecorded suppression is never equivalent to conformance.

6.2 Risk-based changes without self-approval

The `aab diff` operation should report **tightening**, **loosening**, **neutral**, or **indeterminate**, plus affected components, rules, and evidence obligations. For example, adding a forbidden import edge to a prohibition list is usually tightening, while removing such a prohibition is loosening. Changing a rule's scope or a component mapping can be semantically indeterminate, and adding a component may alter trust boundaries or policy coverage.

Classification is **advisory impact analysis, not authorization**. The default is: **every change to a signed baseline requires a newly authorized baseline**. Organizations may permit streamlined approval of low-risk cases using a signed delegation policy and trusted automation, but never allow the code-writing agent to approve its own changes. Loosening a blocking constraint or weakening coverage must always require explicit review by an appropriately authorized person; unknown classifications use the higher review tier. This preserves the benefit of proportionate governance without allowing "neutral" labels to become an approval bypass.

7. The reference validator and its verification contract

The validator is the independent enforcement point; adapters execute specific checks. Its proposed steps are:

1. **Parse and validate** the versioned package using JSON Schema Draft 2020-12 and semantic checks, rejecting duplicate identifiers or malformed rule references [13].
2. **Resolve and index** components, relationships, rule scopes, referenced files, and the canonical package manifest.
3. **Authenticate approval:** recompute the package digest, verify the external signed attestation, bind its identity to trusted authorization policy, and reject an unapproved candidate baseline.
4. **Account for artifacts:** enumerate eligible files and required generated artifacts; detect exclusions, ambiguities, missing assignments, and mapping changes.
5. **Bind adapter capability:** validate the adapter version/digest, resolution policy, supported language/configuration types, and required source coverage.
6. **Evaluate rules:** run isolated, versioned adapters; retain their observations and limitations; do not execute untrusted PR-provided validator code as part of the trust root.
7. **Emit evidence and apply gates:** record source revision, baseline digest, checker identity, coverage, findings, and errors; apply protected L1/L2 policy.

Illustrative CLI shape:

```
aab validate architecture/
aab verify-baseline architecture/ --trust-policy /trusted/policy.json
aab check architecture/ --source . --adapter-lock /trusted/adapters.lock
aab diff --base approved-ref --head candidate-ref
aab generate-instructions architecture/ --output AGENTS.md
```

```
aab verify-instructions AGENTS.md --baseline approved-ref
aab report --format json
```

These are **proposed commands**, not installed software. The architecture package MUST NOT be allowed to redefine the trusted CLI, gate logic, or approval policy used to validate itself.

7.1 Outcome semantics and fail-closed gates

Result	Meaning	Automatic AAB-L2 gate treatment for blocking rule
`conformant`	Declared checker completed and found no violation **within its stated scope**.	Permit this rule only if coverage and trust prerequisites pass.
`violated`	A supported checker found a counterexample.	Block.
`unverified`	Applicability, mapping, capability, or observation was insufficient.	Block.
`not_applicable`	Rule was **provably** outside the declared change/target scope under a documented applicability criterion.	Permit only when scope proof and coverage accounting are valid.

`execution_error` is a separate infrastructure outcome, not a fifth assertion about system architecture. Missing adapter executables, crashes, corrupted input, invalid signatures, or a failed trusted verifier stop the L2 gate. For a required blocking rule, absence of evidence is not evidence of conformance. L1 may provide informative reports without claiming source-code conformance.

A portable evidence record should include at minimum: `schema_version`, `blueprint_id`, `baseline_digest`, `approval_attestation_ref`, `source_revision`, `rule_id`, `rule_type`, `result`, `coverage`, `adapter_id`, `adapter_version`, `adapter_binary_digest`, `resolution_policy`, `time`, `finding_refs`, and an independent `execution_status`. Results must identify any excluded or unmapped files and whether the check examined only the proposed diff or the entire eligible baseline.

7.2 A precise account of determinism

AAB can require **deterministic validation of the same declared inputs**, not omniscient or globally deterministic program analysis. Two runs should agree given the same schema version, blueprint bytes, source tree, adapter binary/version, resolution policy, environment assumptions, and any controlled generated outputs. A Java bytecode analyzer and a Python/TypeScript import resolver have different observation boundaries. Dynamic imports and runtime module loading remain possible blind spots. The evidence format must expose these differences so consumers cannot interpret unlike checks as equivalent [10].

The initial interoperability suite should include fixtures for imports via aliases, re-exports, dynamic module expressions, empty and overlapping path globs, regenerated source files, missing components, configuration selectors with zero matches, and adapter failures. Checkers MUST report limitations when they cannot construct the agreed graph or parsed configuration view.

7.3 Prevent generated `AGENTS.md` drift

Agent-facing instructions are useful but subordinate to the signed baseline. AAB-generated `AGENTS.md` should contain a machine-readable header identifying the `blueprint_id`, `baseline_digest`, and instruction generator version. Trusted CI recomputes the approved baseline digest, validates the instruction header, and compares the generated content with the pinned generator's output. A stale or hand-edited summary is flagged as `instructions_stale`; for L2 workflows that require generated instructions, it is blocking.

Importantly, generated instructions **do not need authority to enforce architecture**. Even if an agent ignores `AGENTS.md`, independent CI still evaluates binding rules against the signed baseline. Instructions are advisory context; contracts and trusted gate configuration determine compliance.

8. Conformance profiles, security, and practical adoption

AAB-L1 (Described) requires a structurally valid blueprint, stable IDs, associated views and ADR references, explicit model scope, and a clear statement of which rules are informative versus binding. It improves documentation but makes no claim of verified implementation conformance.

AAB-L2 (Governed) adds authenticated authorization of the baseline; blocking-rule capability checks; complete accounting of eligible artifacts; independent adapter execution; trustworthy evidence; protected gate policy; and fail-closed handling of violations, unresolved blocking obligations, and verifier errors. An AAB-L2 outcome is **conformance to the declared and checked AAB obligations under a specific baseline**, not proof that the entire system is safe, correct, or well architected.

A future L3 profile may define stronger assurance for selected security, reliability, and safety concerns, but should be informed by field evidence rather than promised by 0.1.

The trust model must explicitly address malicious or simply mistaken repository changes; compromised developer credentials; altered or replayed attestations; unauthorized changes to CI configuration; compromised adapters; and prompt injection through repository content. Protected branches and CODEOWNERS-style controls can help operationally, but are not substitutes for independently verified authorization. Similarly, signing provides provenance and integrity, not semantic correctness. SLSA and software attestation practices provide useful complementary models [15, 19, 20].

Adoption should start small. Teams should identify a handful of high-value architectural boundaries, map code comprehensively within a bounded source tree, select one checkable relation, and exercise deliberate violation cases. The specification should reward honest *unverified* outcomes rather than pressure teams into declaring unsupported rules "green."

9. Related work: what AAB builds on

Architecture description and decisions. The 4+1 views are the communication foundation; ISO/IEC/IEEE 42010 provides architecture-description concepts; ISO/IEC 25010 provides product-quality characteristics; ADRs describe motivations and trade-offs; C4/Structurizr make models understandable and maintainable [1-3, 5, 6]. AAB proposes a checkable governance profile, not an alternative diagramming notation.

Agentic specification workflows. GitHub Spec Kit structures constitution, specification, planning, tasks, implementation, and convergence [7]. OpenSpec maintains change proposals, delta specifications, design artifacts, and tasks [8]. Kiro offers structured specification artifacts [9], and `AGENTS.md` is an intentionally lightweight agent-instruction format [16]. These tools can already support architecture checks and should be regarded as complementary, not as if they offer no architectural governance. The intended AAB contribution is a portable **signed architecture baseline and evidence protocol**, usable across these workflows and independent of any agent's own report.

Executable architecture and policy. ArchUnit can verify Java dependency and layering rules; Conftest evaluates configuration data under Rego policies; architectural fitness functions put selected architectural properties into continuous checks [4, 10, 11]. AAB standardizes how the model links to these mechanisms, what scope each check claims, and how coverage failure is represented.

Artifact integrity and software supply-chain attestation. RFC 8785 addresses canonical JSON bytes; in-toto defines a statement shape connecting a subject digest with a predicate; Sigstore provides signature-verification workflows; SLSA provides complementary provenance patterns [18-20, 15]. AAB reuses these building blocks for *approval of architectural intent*, while acknowledging that verification of an artifact's origin does not prove architectural quality.

Emerging AI-native engineering research. Konrad and colleagues discuss implicit design decisions made by coding agents [14]; Alenezi proposes a specification-governance model for AI-native engineering

[17]. Both are 2026 preprints and should be cited as emerging proposals, not as conclusive validation of AAB's claimed benefits. The open contribution remains **interoperability at the boundary between architectural approval, implementable constraints, and bounded evidence**.

10. Implementation roadmap and empirical validation

AAB 0.1 should be developed as a small, testable reference project, using established libraries rather than building an architecture authoring IDE or general-purpose policy language.

Milestone	Concrete deliverable	Proposed exit criterion
M1: Semantic core	Terminology, three typed rules, applicability and scope fixtures.	At least 3 independent reviewers classify 50+ representative cases with >=95% raw agreement and target multi-rater kappa >=0.80; resolve all consequential disagreements before release.
M2: Canonical package	Published JSON Schema, hash manifest, canonicalization and malicious-input fixtures.	Two independent parsers agree on valid/invalid cases and produce identical digests for published test vectors.
M3: Validator	CLI, import dependency adapter, config adapter, full artifact inventory.	Seeded forbidden imports, unmapped files, overlaps, and vacuous required dependencies are correctly rejected.
M4: Authorization	Signed external baseline statement, independently managed trust policy, candidate diff classifications.	Editing rules, hashes, generated instructions, or approvals in an untrusted PR cannot bypass the L2 gate.
M5: Evidence and interoperability	Portable JSON evidence format, coverage reports, pinned adapter contract.	Repeated runs agree on the same declared inputs; unsupported capabilities and execution errors never produce a false pass.
M6: Evaluation	Multi-agent and multi-repository controlled study.	Publish violation-detection recall, false-positive rate, uncovered-rule rate, reviewer effort, latency, and amendment burden with uncertainty estimates.

The inter-rater thresholds in M1 are **research targets proposed for discussion**, not empirical results. Agreement should be reported together with class prevalence, disagreement categories, and the rationale for the selected statistic; kappa alone can be misleading when one classification dominates.

A prospective comparative study should use three conditions on architecture-sensitive change requests: **(A) prompting only**, **(B) spec-driven development without AAB**, and **(C) spec-driven development plus AAB**. Tasks, repositories, model versions, context budgets, and resources should be controlled as far as possible. Seed known violations, including new unmapped directories and unauthorized blueprint changes. Independent reviewers and predefined oracles, not the implementing agents themselves, should score architecture deviations. Measure successful tasks, total time, governance overhead, missed violations, false positives, and traceability quality. This study is necessary before claiming effectiveness.

11. Remaining limitations and research questions

A signed blueprint may embody a poor design; authorization validates a decision, not its wisdom. An accurate static dependency graph does not guarantee correct runtime isolation. Configuration evaluation against static manifests may not capture mutating cluster admission or operational drift. Root-selection policies can still omit entire ecosystems unless independently reviewed. A policy server or identity system can be compromised, and signature-valid decisions can be unauthorized if the authorization policy is weak.

The proposal therefore needs a formal threat model, independently reviewed capability mappings, test fixtures covering attacker-controlled repository changes, and well-defined change-review duties. It must also limit adoption overhead: a small number of precise rules is more valuable than a large and unmaintainable set of aspirational constraints.

Finally, the hardest interoperability problems are semantic, not syntactic: different language analyzers construct different dependency graphs; different CI environments observe different artifacts; and risk classification is not always monotonic. AAB should publish explicit incompatibilities rather than pretend every verifier implements the same concept of "dependency."

12. Conclusion

Agentic coding makes software changes cheap to propose and fast to implement. It does not remove the responsibility to make deliberate, durable architectural decisions. AAB answers this challenge by making approved architecture a **portable contract with separately controlled authorization and honest, scoped conformance evidence**.

This revised AAB 0.1 proposal strengthens its most important guarantees: signed approvals cannot be fabricated merely by editing a file; source files cannot silently escape a declared coverage boundary; rule semantics are deliberately narrow; generated agent instructions are checked for freshness; and evidence distinguishes violations from unknowns and infrastructure failures. Risk-based review remains possible without giving agents approval authority.

The appropriate next step is an open technical specification and independent reference validator, accompanied by malicious-input fixtures, interoperability tests, and controlled evaluations on real projects. AAB's value should ultimately be demonstrated in fewer undetected architectural violations and clearer accountability, not assumed from the presence of architecture documents or signatures.

References

[1] Kruchten, P. (1995). "The 4+1 View Model of Architecture." *IEEE Software*, 12(6), 42-50. https://doi.org/10.1109/52.469759.

[2] ISO/IEC/IEEE. (2022). *ISO/IEC/IEEE 42010:2022 — Software, systems and enterprise: Architecture description*. https://www.iso.org/standard/74393.html.

[3] ISO/IEC. (2023). *ISO/IEC 25010:2023 — Systems and software engineering: SQuaRE — Product quality model*. https://www.iso.org/standard/78176.html.

[4] Ford, N., Parsons, R., & Kua, P. (2017). *Building Evolutionary Architectures: Support Constant Change*. O'Reilly Media. https://www.oreilly.com/library/view/building-evolutionary-architectures/9781491986356/.

[5] Nygard, M. (2011, November 15). "Documenting Architecture Decisions." https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions.

[6] Brown, S. (n.d.). *The C4 Model* and *Structurizr Documentation*. https://c4model.com/ and https://docs.structurizr.com/.

[7] GitHub. (2026, accessed October 12). *Spec Kit: Specification-driven development documentation*. Official documentation and source repository.

[8] Fission AI. (2026, accessed October 12). *OpenSpec: Spec-driven change workflows*. https://github.com/Fission-AI/OpenSpec.

[9] Kiro. (2026, accessed October 12). *Kiro Specs documentation*. https://kiro.dev/docs/specs/.

[10] ArchUnit contributors. (2026, accessed October 12). *ArchUnit User Guide*. https://www.archunit.org/userguide/html/000_Index.html.

[11] Open Policy Agent contributors. (2026, accessed October 12). *Conftest: Testing structured configuration*. Source repository.

[12] Bradner, S. (1997). *RFC 2119: Key words for use in RFCs to Indicate Requirement Levels*; Leiba, B. (2017). *RFC 8174: Ambiguity of*

Uppercase vs Lowercase. https://www.rfc-editor.org/rfc/rfc2119 and https://www.rfc-editor.org/rfc/rfc8174.

[13] JSON Schema contributors. (2022). *JSON Schema Draft 2020-12*. https://json-schema.org/draft/2020-12/.

[14] Konrad, P. M., Adam, T. L., Terrenzi, R., & Ayvaz, S. (2026). "Architecture Without Architects: How AI Coding Agents Shape Software Architecture." *arXiv preprint* arXiv:2604.04990. https://arxiv.org/abs/2604.04990.

[15] SLSA contributors. (2026, accessed October 12). *SLSA v1.2: Provenance*. https://slsa.dev/spec/v1.2/provenance.

[16] AGENTS.md contributors. (2026, accessed October 12). *AGENTS.md: A simple format for coding-agent instructions*. https://agents.md/.

[17] Alenezi, M. (2026). "Specification-Driven Development as the Foundation of AI-Native Enterprise Software Engineering." *arXiv preprint* arXiv:2607.16680. https://arxiv.org/abs/2607.16680.

[18] Rundgren, A., Jordan, B., & Erdtman, S. (2020). *RFC 8785: JSON Canonicalization Scheme (JCS)*. https://www.rfc-editor.org/rfc/rfc8785.html.

[19] in-toto contributors. (2026, accessed October 12). *in-toto Attestation Framework: Statement v1*. https://github.com/in-toto/attestation/blob/main/spec/v1/statement.md.

[20] Sigstore contributors. (2026, accessed October 12). *Cosign: Verify a signed blob*. https://github.com/sigstore/cosign/blob/main/doc/cosign_verify-blob.md.